

Manual Técnico del Sistema ReclamaSeguro (Liclam Lex Insurance)

Versión del Documento: 1.0

Fecha: 17 de Diciembre de 2025

Proyecto: ReclamaSeguro

1. Introducción y alcance

Este documento técnico describe la arquitectura, componentes, configuración y procedimientos de despliegue del sistema **ReclamaSeguro**. La plataforma está diseñada para la gestión automatizada de reclamaciones de seguros, consultas legales y liquidaciones, facilitando la interacción entre usuarios finales y administradores a través de una interfaz web moderna y una API robusta.

1.1 Objetivo del sistema

Proporcionar una plataforma segura y escalable que permita:

- Registro y autenticación de usuarios.
- Creación y seguimiento de consultas y reclamaciones (Wizards).
- Gestión administrativa de casos.
- Procesamiento de pagos y carga de documentación.

2. Arquitectura del sistema

El sistema sigue una arquitectura de **Cliente-Servidor** desacoplada, utilizando el patrón **MVC (Modelo-Vista-Controlador)** en el backend y una **SPA (Single Page Application)** en el frontend.

2.1 Diagrama de tecnologías

- **Frontend:** Angular (v16+), TypeScript, HTML5, CSS3.
- **Backend:** Java Spring Boot, Maven.
- **Base de Datos:** Relacional (SQL), gestionada vía JPA/Hibernate (Inferido por Repositories).
- **Comunicación:** API RESTful (JSON).
- **Seguridad:** JWT (JSON Web Tokens), Spring Security, Rate Limiting.

3. Subsistema Frontend (Cliente Web)

El frontend está construido sobre el framework **Angular**. La estructura del proyecto sugiere una aplicación modular orientada a componentes.

3.1 Estructura del Proyecto

La aplicación reside en `src/app` y se divide en módulos funcionales clave:

Componente/Módulo	Descripción Funcional
Admin	Panel de administración para gestionar usuarios y reclamaciones (<code>admin.component</code>).
Auth & Login/Register	Manejo de sesión. <code>login-register</code> gestiona la entrada y <code>auth_consultation</code> parece ser un flujo específico para consultas autenticadas.
Wizard & Wizard Registered	Núcleo de la aplicación. Guía paso a paso para que el usuario ingrese su reclamación. Existe una variante para usuarios ya registrados.
Claim & Request	Gestión y visualización de una reclamación (<code>claim</code>) y la solicitud inicial (<code>request</code>).
Document	Módulo <code>upload-documents</code> encargado de la subida de archivos (PDFs, imágenes) como evidencia.
Liquidation	Visualización o cálculo de liquidaciones de seguros.
Payment	Pasarela de pagos o selección de método de pago (integración visual con <code>assets</code> de Visa/Mastercard/Paypal).
User	Perfil y gestión de datos del usuario final.
Notification	Sistema de alertas y feedback al usuario.

3.2 Servicios (Capa de Lógica)

Ubicados en `src/app/services`, encapsulan la comunicación con el Backend:

- **ApiService:** Centraliza las llamadas HTTP (GET, POST, PUT, DELETE) hacia los endpoints de Spring Boot.
- **NotificationService:** Gestiona la visualización de mensajes (toasts, alertas) para mejorar la UX.
- **SharedDataService:** Facilita la comunicación de estado entre componentes no relacionados (State Management básico).

3.3 Seguridad y Routing

- **Guards (`src/app/guards`):**
 - **AuthGuard:** Protege rutas que requieren inicio de sesión.

- AdminGuard: Restringe el acceso a las rutas de administración (/admin).
- **SSR**: La presencia de main.server.ts y app.config.server.ts indica que la aplicación está configurada para **Angular Universal (Server-Side Rendering)**, mejorando el SEO y la carga inicial.

4. Subsistema Backend (API Server)

El backend está desarrollado en **Java** utilizando **Spring Boot**. El proyecto se gestiona con **Maven** (pom.xml).

4.1 Estructura de Paquetes (com.liclam.lexinsurance)

4.1.1 Controladores (controller)

Exponen los endpoints de la API REST:

- **AuthController**: Endpoints para /login, /register, y renovación de tokens.
- **ConsultationController**: Maneja el ciclo de vida de las reclamaciones (crear, leer, actualizar).
- **AdminController**: Endpoints privilegiados para obtener estadísticas (AdminStatsDto) y gestión global.

4.1.2 Servicios (service)

Contienen la lógica de negocio:

- **AuthService**: Lógica de validación de credenciales y generación de sesiones.
- **ConsultationService**: Procesa las reglas de negocio de las reclamaciones.
- **EmailService**: Servicio para el envío de notificaciones por correo electrónico (confirmaciones, actualizaciones de estado).

4.1.3 Modelo de Datos y DTOs

- **Entidades (entity)**:
 - User: Representa la tabla de usuarios en la BD.
 - Consultation: Representa la tabla de reclamaciones/consultas.
- **DTOs (dto)**: Objetos de transferencia de datos para desacoplar la API de la BD.
 - RegisterRequest, LoginRequest: Payloads de entrada.
 - AdminStatsDto, AdminConsultationDto: Respuestas optimizadas para el panel de administración.
 - SignatureRequest: Sugiere funcionalidad de firma digital o validación de documentos.

4.1.4 Capa de acceso a datos (repository)

Interfaces que extienden de JpaRepository para interactuar con la base de datos:

- UserRepository
- ConsultationRepository

4.2 Configuración y seguridad

- **SecurityConfig.java:** Configuración de Spring Security. Define cadenas de filtros, CORS, y protección CSRF (si aplica).
- **RateLimitFilter.java:** Implementación de seguridad avanzada para prevenir ataques de fuerza bruta o DoS, limitando el número de peticiones por IP/Usuario.
- **CryptoService.java:** Utilidad para encriptación/descriptación, usada para datos sensibles antes de persistir o para firmas.

5. Requisitos de instalación y despliegue

5.1 Prerrequisitos

- **Node.js:** v18.x o superior.
- **Java JDK:** v17 o superior.
- **Maven:** v3.8+ (o usar el wrapper mvnw incluido).
- **Base de Datos:** MySQL o PostgreSQL (Configurar en application.properties).

5.2 Despliegue del Backend

1. Navegar a la carpeta del backend.
2. Configurar la conexión a la base de datos en src/main/resources/application.properties.
3. Ejecutar el build:
./mvnw clean install
4. Iniciar la aplicación:
java -jar target/ReclamaSeguro_BACKEND-0.0.1-SNAPSHOT.jar

5.3 Despliegue del Frontend

1. Navegar a la carpeta del frontend.
2. Instalar dependencias:
npm install
3. Compilar para producción:
ng build --configuration production
4. Los archivos generados en dist/ deben ser servidos por un servidor web (Nginx, Apache) o mediante el servidor SSR de Node.js si se usa Universal.

6. Mantenimiento y Escalabilidad

- **Gestión de Errores:** El backend debe implementar un `@ControllerAdvice` (recomendación estándar) para manejar excepciones globales.
- **Logs:** Configuración de Logback o Log4j2 para monitorear la actividad del sistema (visible en `LiclamApplication`).
- **Testing:**
 - Frontend: Pruebas unitarias con Jasmine/Karma (*.spec.ts).
 - Backend: Pruebas de integración con JUnit (`LiclamApplicationTests.java`).